PILOT — Training the `action` vs `noaction` data (teammate runbook)

Goal for whoever picks this up: take the 8 CSV files captured on 2026-07-17 (`~/usrp-projects/data/pilot/`), turn them into a trained model, and report **one honest number** — the per-window test accuracy on an *unseen subject*. Then hand back the two `.pt` model files and that number.

This is a **binary** problem (`action` vs `noaction`), so **chance = 50%**. It is a throwaway pilot to prove the whole chain works before the full 7-class collection.

You do **not** write any code. Every tool already exists in `~/usrp-projects/USRP_lib/csi_tools/` . This doc explains *what each step does and why*, then gives the *exact command* to run. Follow Parts A → B → C → D in order.

Read `CONTEXT.md` and `PILOT_ACTION_VS_NOACTION.md` (the capture-side plan) once for background — this runbook is the training-side companion.

Prerequisites (check once before Part B)

Repo: you need the `USRP_lib` repo present locally — specifically `USRP_lib/csi_tools/` (`segment_windows.py` , `make_images.py` , `train_imgfi.py` , `check_capture.py`). This `.md` only gives commands; it doesn't carry those scripts.

Data: the 8 CSVs under `data/pilot/` (the `_check.png` files that ship alongside them are leftover eyeball-check plots from capture day — not needed for training, ignore them).

Python packages:

```
pip install numpy pandas torch Pillow matplotlib
```

Do **not** install `torchvision` — it's deliberately excluded from this project (`train_imgfi.py` only uses `torch` , `numpy` , and `PIL`).

GPU: a CUDA GPU is picked up automatically if present. Without one, training falls back to CPU — it will still work, just much slower than the "~10–20 min per run" quoted in Part C.

The data you are training on

8 captures, 2 subjects, 4 takes each. `action` = the person is moving; `noaction` = the person is present but still (the model must learn *motion*, not *presence*).

Class	Take	Subject	File	rows/s	duration
`action`	walk	rishiikesh	`P01_action_rishiikesh_20260717_walk.csv`	77.7	91.6 s
`action`	mixed	rishiikesh	`P01_action_rishiikesh_20260717_mixed.csv`	56.1	95.9 s
`action`	walk	amrith	`P01_action_amrith_20260717_walk.csv`	73.0	91.9 s
`action`	mixed	amrith	`P01_action_amrith_20260717_mixed.csv`	73.5	107.9 s
`noaction`	stand	rishiikesh	`P01_noaction_rishiikesh_20260717_stand.csv`	62.8	89.8 s
`noaction`	sit	rishiikesh	`P01_noaction_rishiikesh_20260717_sit.csv`	51.5	90.9 s
`noaction`	stand	amrith	`P01_noaction_amrith_20260717_stand.csv`	65.6	92.1 s

Class	Take	Subject	File	rows/s	duration
`noaction`	sit	amrith	`P01_noaction_amrith_20260717_sit.csv`	71.0	91.8 s

> **The one number that matters about this data:** capture rate is **51–78 rows/s**, not the ~100 rows/s the pipeline defaults assume. Everything still works, but you must pass `--rate 50` at the segmentation step (Part B) or the slow files will silently throw away windows. This is the single most important correction in this doc.

Note on the `mixed` take (a capture-day correction)

The `mixed` take was recorded as a continuous loop of: `squat → bend to pick up a water bottle → stand → drink → bend to put it back → stand → 2 side-steps left/right` — about **18–20 s per loop**, so amrith's ran ~108 s.

For **this binary pilot none of that internal structure matters**: the whole `mixed` file is one continuous `action` capture, and the segmenter labels the *entire* file `action`. It is deliberately a **throwaway** — because squat/bend/drink are blended together with no per-rep timestamps, this file **cannot** be relabeled into separate classes for the future 7-class work. Don't try. (Same reasoning as capture-guide §9.)

Part A — The technique, in one page (understand this before running anything)

The pipeline has four stages:

```
capture CSV (52 complex subcarriers per WiFi packet)
      │  segment_windows.py   → amplitude, cut into 2 s windows
      ▼
amplitude windows      (52 subcarriers × 100 samples, .npy)
      │  make_images.py       → one recurrence image PER subcarrier
      ▼
RT images              (224×224 grayscale PNG, ×52 per window)
      │  train_imgfi.py       → small 3-conv CNN
      ▼
ImgFi CNN  →  per-window prediction (majority vote of the 52 images)
```

Why amplitude only, never raw phase. The USRP is not clock-synced to the MacBook, so every packet carries random carrier/sampling/timing offset that scrambles the *phase*. The magnitude $|H| = \sqrt{Re^2 + Im^2}$ is immune to that. The movement information lives in how the amplitude fluctuates, so we keep amplitude and discard phase.

Why turn a time-series into an image (the core ImgFi idea). For one subcarrier's amplitude series $x = (x_1 \dots x_n)$ we build a **Recurrence Transformation (RT)** image:

$$RT[i, j] = | x_i - x_j |$$

Every pixel is the difference between the signal at two moments *i* and *j*. So the basic unit a CNN sees is already *"how much did the channel change between any two instants"* — which is exactly what a human movement is. Time order is preserved along the diagonal (top-left = start, bottom-right = end). The ImgFi paper compared 5 ways of imaging CSI (RP/RT, GASF, GADF, MTF, STFT); **RT scored highest (~99.8%) and is the cheapest to compute**, which is why we use it.

Why one image per subcarrier (×52). Each of the 52 subcarriers is a slightly different "view" of the same movement (different frequency → different multipath sensitivity). Treating each as its own training image multiplies the dataset ×52 for free — that's what lets a tiny CNN train from scratch on only a couple of minutes of capture.

The honest-split rule — the thing that keeps the result real. All 52 images of one window come from the **same physical 2 seconds**. If you split train/test by shuffling images randomly, near-identical copies of the same event land on both sides and you get a meaningless ~99%. So the split is **by subject** (train on one person, test on the other),

never random. `train_imgfi.py` refuses to run without a subject/match flag — that's on purpose, don't work around it.

Two accuracies get printed — only one counts.

- **per-image** — accuracy over individual subcarrier images. *Diagnostic only.*
- **per-window** — the 52 images of a window vote, majority wins. **This is the real system accuracy** ("the person moved → did we say `action`?"). Report this one.

Part B — Preprocess the 8 CSVs

B0 — Set up your shell (once)

```
cd ~/usrp-projects/USRP_lib/csi_tools
D=~/usrp-projects/data/pilot     # where the 8 CSVs live and outputs go
S1=rishiikesh
S2=amrith
DATE=20260717
```

B1 — (recommended) confirm each capture is healthy first

Why: cheaper to catch a bad file now than after images + training. `check_capture.py` prints rows/s, gaps, and flags NaN/zero/constant subcarriers.

```
for f in D/P01_*_{DATE}_*.csv; do echo "=== f ==="; python3 check_capture.py "f"; done
```

Expect every file to say `RATE GATE (>= 50 rows/s): PASS`, `NaN values: 0`, `All-zero rows: 0`. (Rate below 100 is fine and expected here — Part B2 handles it.)

B2 — Segment each take into 2 s amplitude windows

Why `--rate 50`: the segmenter resamples each 2 s window onto a uniform grid of `rate × 2` samples and skips any window that has fewer than half that many *real* packets. At the default `--rate 100` a window needs ~100 packets; the slow files here (sit 51.5/s, mixed 56/s) don't clear that reliably and windows get dropped. `--rate 50` makes each window `52 × 100` and only needs ≥50 packets per 2 s — every file passes cleanly. **Use the same `--rate 50` for all 8 files** so every window has the same shape.

Why the `--action` value is `action` / `noaction` (not walk/sit): in auto-chunk mode the output class folder comes straight from `--action`. So **both `walk` and `mixed` go under `--action action`**, and **both `stand` and `sit` go under `--action noaction`**.

```
# action class (walk + mixed, both subjects)
python3 segment_windows.py D/P01_action_{S1}_DATE_walk.csv --rate 50 --action action --subject S1 --out-dir $D/windows
python3 segment_windows.py D/P01_action_{S1}_DATE_mixed.csv --rate 50 --action action --subject S1 --out-dir $D/windows
python3 segment_windows.py D/P01_action_{S2}_DATE_walk.csv --rate 50 --action action --subject S2 --out-dir $D/windows
python3 segment_windows.py D/P01_action_{S2}_DATE_mixed.csv --rate 50 --action action --subject S2 --out-dir $D/windows

# noaction class (stand + sit, both subjects)
python3 segment_windows.py D/P01_noaction_{S1}_DATE_stand.csv --rate 50 --action noaction --subject S1 --out-dir $D/windows
python3 segment_windows.py D/P01_noaction_{S1}_DATE_sit.csv --rate 50 --action noaction --subject S1 --out-dir $D/windows
python3 segment_windows.py D/P01_noaction_{S2}_DATE_stand.csv --rate 50 --action noaction --subject S2 --out-dir $D/windows
python3 segment_windows.py D/P01_noaction_{S2}_DATE_sit.csv --rate 50 --action noaction --subject S2 --out-dir $D/windows
```

Each take (~90 s) → **~45 windows**; amrith's longer `mixed` → ~54. Watch the `Saved N windows (M skipped)` line at the end of each call: **a few skipped is normal, more skipped than saved means a rate/gap problem** in that capture (re-check it with B1).

Output lands in *D/windows/action/*.npy* and `D/windows/noaction/*.npy`. Filenames carry the subject (`_rishiikesh_` / `_amrith_`) and take type (`walk` / `mixed` / `stand` / `sit`) — that's what the training splits key off later, so don't rename them.

B3 — Balance check

Why: if one class has far more windows than the other, accuracy gets biased. Count them:

```
for d in D/windows/*/; do echo "$d: $(ls "$d" | wc -l)"; done
```

Expect roughly **~370 windows total, ~52%** `action` (slightly more `action` only because amrith's `mixed` take ran long). That's close enough — no capping needed. If one class ever comes out >20% larger, delete the excess `.npy` from the bigger class.

Optional data-doubling: add `--step 1.0` to the B2 commands to emit overlapping windows (one every 1 s instead of every 2 s), roughly doubling the count to ~740. It stays honest because train/test is still split by subject. Do this only if you want more data; the headline number should come from the non-overlapping run above.

B4 — Convert windows to RT images

Why: this is the RT recurrence transform from Part A — one 224×224 grayscale PNG per subcarrier per window, min-max normalized. Rerunning is safe (it skips images that already exist).

```
python3 make_images.py D/windows --out D/dataset
```

Result: *D/dataset/action/*.png* and `D/dataset/noaction/*.png` — about **~19 000 PNGs total (~9 000+ per class)** from ~370 windows × 52 subcarriers.

Part C — Train and read the result

C1 — The two honest runs (leave-one-subject-out)

Why two runs: with 2 subjects, the honest test is to train on one person and test on the *other*, both ways, then **report the worse of the two per-window accuracies**. That is the pilot's number. (Be aware: train-on-1 / test-on-1 is a genuinely hard, high-variance test — that's inherent to a 2-person pilot, not a bug.)

No code change for binary: `train_imgfi.py` reads the class list from the folder names, sees `['action', 'noaction']`, and builds a 2-way classifier automatically.

```
# train on rishiikesh, test on amrith
python3 train_imgfi.py D/dataset --test-subject S2 --out D/imgfi_pilot_test-{S2}.pt

# train on amrith, test on rishiikesh
python3 train_imgfi.py D/dataset --test-subject S1 --out D/imgfi_pilot_test-{S1}.pt
```

Each run prints, per epoch, a rising `val_win` column, then a final block:

```
TEST  per-image acc 0.9xx   per-window acc 0.9xx
Confusion matrix (rows=true, cols=pred):
```

Uses the CUDA GPU automatically; ~10–20 min per run. **Record the `per-window acc` from each run.**

C2 — What the number means (pass / fail)

Take the **lower** of the two per-window accuracies and read it here:

Per-window (worse of the two runs)	Meaning	Next step
≥ 95%	Pipeline + signal are healthy	Green light → the full 7-class plan can begin
85–95%	Something is weak	Run the diagnostics in C3, fix, re-collect just the weak take
< 85%	Signal problem (rate/gain/geometry)	Do **not** start multi-class; go back to the capture-side wiggle/rate gates
suspiciously = 100.0% on everything	Fine for walk-vs-still, but **double-check you used `--test-subject`** (not a random split) before trusting it	—

C3 — Diagnostics (only if you landed in 85–95%)

The take type is baked into every filename, so `--test-match` can hold out just one take type as the test set and tell you how well that specific motion is recognized:

```
python3 train_imgfi.py $D/dataset --test-match mixed   # weak/small motions — the likely culprit
python3 train_imgfi.py $D/dataset --test-match walk    # strong motion — should be ~100%
python3 train_imgfi.py $D/dataset --test-match sit     # stillness, seated
python3 train_imgfi.py $D/dataset --test-match stand   # stillness, standing
```

Typical finding: `walk` ≈ 100% but `mixed` low → the small drink/bend motions are near the noise floor. That points at capture (exaggerate motions, confirm the subject spot really blocks the direct path, re-check gain), not at the model.

C4 — What to hand back

- The two model files: `imgfi_pilot_test-amrith.pt` and `imgfi_pilot_test-rishiikesh.pt`.
- **Both** per-window accuracies, and the **worse one as the headline number**.
- The two confusion matrices (printed, and saved as `*_confusion.csv` next to each `.pt`) — for binary, look at how much `action` and `noaction` get mixed up.

Part D — Troubleshooting

Symptom	Cause → fix
`Saved N windows (M skipped)` with M > N	Capture rate too low for the chosen `--rate`. You are already at `--rate 50`; re-run `check_capture.py` on that file — if it's genuinely slow/gappy, the capture is bad, not the segmenter
`train_imgfi.py` errors `need --test-subject or --test-match`	Intentional — random splits are blocked. Pass one of the flags
`split produced empty set (test=0 …)`	The subject/match string isn't in the filenames. Check spelling: it must match `_rishiikesh_` / `_amrith_`, or `walk` / `mixed` / `stand` / `sit`
Per-window acc looks *too* perfect (100%)	Confirm you used `--test-subject`, not a random shuffle (the script forbids random, but double-check the flag)

Symptom	Cause → fix
Per-window acc < 85% on the unseen subject	Eyeball the images: open a few `dataset/action/*.png` next to `dataset/noaction/*.png`. If **you** can't see a texture difference, the CNN can't either → the problem is the *signal* (rate/gain/geometry), go back to the capture-side gates. Only if the images clearly differ is it worth trying more epochs or a larger model
`No PNGs found under …` at training	You skipped Part B4, or pointed at `windows/` instead of `dataset/`

One-glance checklist

```
[ ] B1  check_capture.py on all 8 → PASS, no NaN/zero
[ ] B2  8 × segment_windows.py with --rate 50 → windows/action + windows/noaction
[ ] B3  window counts roughly balanced (~370 total, ~52% action)
[ ] B4  make_images.py → dataset/action + dataset/noaction (~19k PNGs)
[ ] C1  train twice: --test-subject amrith, then --test-subject rishiikesh
[ ] C2  headline = the WORSE per-window acc, judged on the ≥95 / 85–95 / <85 bar
[ ] C4  hand back: 2 .pt files + both numbers + confusion matrices
```